

STEEL INDUSTRY ENERGY CONSUMPTION PREDICTION

AI/ML Internship – Week 2 Report

Submitted By: Farhan

Institute: ITSimplera Institute

Team Lead: Reyan Khan

Submission Date: 11-7-2026



1. Introduction

The objective of this project is to build a machine learning model that can accurately predict energy consumption in the steel manufacturing industry. Energy management is one of the most important challenges in industrial production because high electricity consumption directly affects production costs and operational efficiency.

In this project, the Steel Industry Energy Consumption dataset was analyzed using various data preprocessing and machine learning techniques. The complete workflow included data loading, data cleaning, exploratory data analysis (EDA), feature engineering, categorical data encoding, feature scaling, and baseline regression modeling.

Several regression algorithms were trained and evaluated to compare their prediction performance. The project provides practical experience in implementing a complete machine learning pipeline using Python and Scikit-learn.

2. Dataset Description

The Steel Industry Energy Consumption dataset contains information related to electricity usage and manufacturing conditions inside a steel production plant. The dataset consists of numerical and categorical variables that describe different operational parameters affecting energy consumption.

The dataset includes production-related variables such as power usage, load type, time information, lagging and leading reactive power, carbon dioxide emissions, and other industrial measurements.

The target variable represents the amount of electricity consumed, while the remaining variables are used as input features for machine learning models.

The dataset is suitable for regression analysis because the target variable contains continuous numerical values.

3. Dataset Overview

After importing the dataset into Python, the first step was to examine its overall structure and quality.

The following operations were performed:

- Loaded the dataset using Pandas.
- Displayed the first five rows using the `head()` function.
- Displayed the last five rows using the `tail()` function.
- Checked the dataset shape.
- Examined column names.
- Identified the data types of all features.
- Generated descriptive statistics using the `describe()` function.
- Verified the overall structure using the `info()` function.

These steps helped in understanding the dataset before starting preprocessing and model development.

Results:

- Dataset Shape
- Number of Rows: 35040
- Number of Columns: 11

Screenshot: (

The screenshot shows a Jupyter Notebook interface with three code cells. The first cell displays the first five rows of a DataFrame using `df.head()`. The second cell shows the dimensions of the DataFrame using `df.shape`, resulting in `(35040, 11)`. The third cell shows the data types of the columns using `df.info()`, indicating it is a `<class 'pandas.core.frame.DataFrame'>`.

	date	Usage_kWh	Lagging_Current_Reactive.Power_kVarh	Leading_Current_Reactive_Power_kVarh	CO2(tCO2)	Lagging_Current_Power_Factor	Leading_Current_Power_Factor	NSM	WeekS
0	01/01/2018 00:15	3.17	2.95	0.0	0.0	73.21	100.0	900	Wee
1	01/01/2018 00:30	4.00	4.46	0.0	0.0	66.77	100.0	1800	Wee
2	01/01/2018 00:45	3.24	3.28	0.0	0.0	70.28	100.0	2700	Wee
3	01/01/2018 01:00	3.31	3.56	0.0	0.0	68.09	100.0	3600	Wee
4	01/01/2018 01:15	3.82	4.50	0.0	0.0	64.72	100.0	4500	Wee

hape the data set

`df.shape`

`(35040, 11)`

n dataset 11 columns and 35040 rows

ext find information of dataset

`df.info()`

`<class 'pandas.core.frame.DataFrame'>`

)

4. Data Quality Analysis

Before applying machine learning algorithms, the dataset was carefully examined to identify any data quality issues. High-quality data is essential for building reliable and accurate predictive models.

The following data quality checks were performed.

4.1 Missing Values

The dataset was inspected to identify missing values in each column using the `isnull().sum()` function.

Missing values can negatively affect the performance of machine learning models if they are not handled properly. Therefore, each feature was examined carefully before preprocessing.

Observation

The missing values were identified and handled appropriately to ensure that the dataset remained clean and suitable for model training.

Screenshot:

(

```
df.describe().T
```

	count	mean	std	min	25%	50%	75%	max
Usage_kWh	35040.0	27.386892	33.444380	0.0	3.20	4.57	51.2375	157.18
Lagging_Current_Reactive.Power_kVarh	35040.0	13.035384	16.306000	0.0	2.30	5.00	22.6400	96.91
Leading_Current_Reactive_Power_kVarh	35040.0	3.870949	7.424463	0.0	0.00	0.00	2.0900	27.76
CO2(tCO2)	35040.0	0.011524	0.016151	0.0	0.00	0.00	0.0200	0.07
Lagging_Current_Power_Factor	35040.0	80.578056	18.921322	0.0	63.32	87.96	99.0225	100.00
Leading_Current_Power_Factor	35040.0	84.367870	30.456535	0.0	99.70	100.00	100.0000	100.00
NSM	35040.0	42750.000000	24940.534317	0.0	21375.00	42750.00	64125.0000	85500.00

check the missing values

```
df.isnull().sum()
```

date	0
Usage_kWh	0
Lagging_Current_Reactive.Power_kVarh	0
Leading_Current_Reactive_Power_kVarh	0
CO2(tco2)	0
Lagging_Current_Power_Factor	0
Leading_Current_Power_Factor	0
NSM	0
WeekStatus	0
Day_of_week	0
Load_Type	0

dtype: int64

)

4.2 Duplicate Values

The dataset was checked for duplicate records using the **duplicated().sum()** function.

Duplicate records may introduce bias into the training data and reduce the quality of predictions.

Observation

Duplicate records, if present, were removed before proceeding with further analysis.

4.3 Data Types

The data type of every column was verified using the **info()** function.

Understanding data types is important because numerical and categorical features require different preprocessing techniques before model training.

The dataset consisted of both numerical and categorical variables, and all data types were verified before preprocessing.

5. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to better understand the dataset and discover meaningful patterns before training machine learning models.

EDA helps identify relationships between variables, detect outliers, understand feature distributions, and support better decision-making during feature engineering.

Several visualizations were created to analyze the data.

5.1 Distribution of Numerical Features

Histograms were generated to examine the distribution of numerical variables.

These visualizations helped determine whether the features followed normal distributions or contained skewed values.

Explanation

The histogram illustrates how the values of each numerical feature are distributed throughout the dataset. It also helps identify skewness and unusual patterns that may require preprocessing.

5.2 Correlation Heatmap

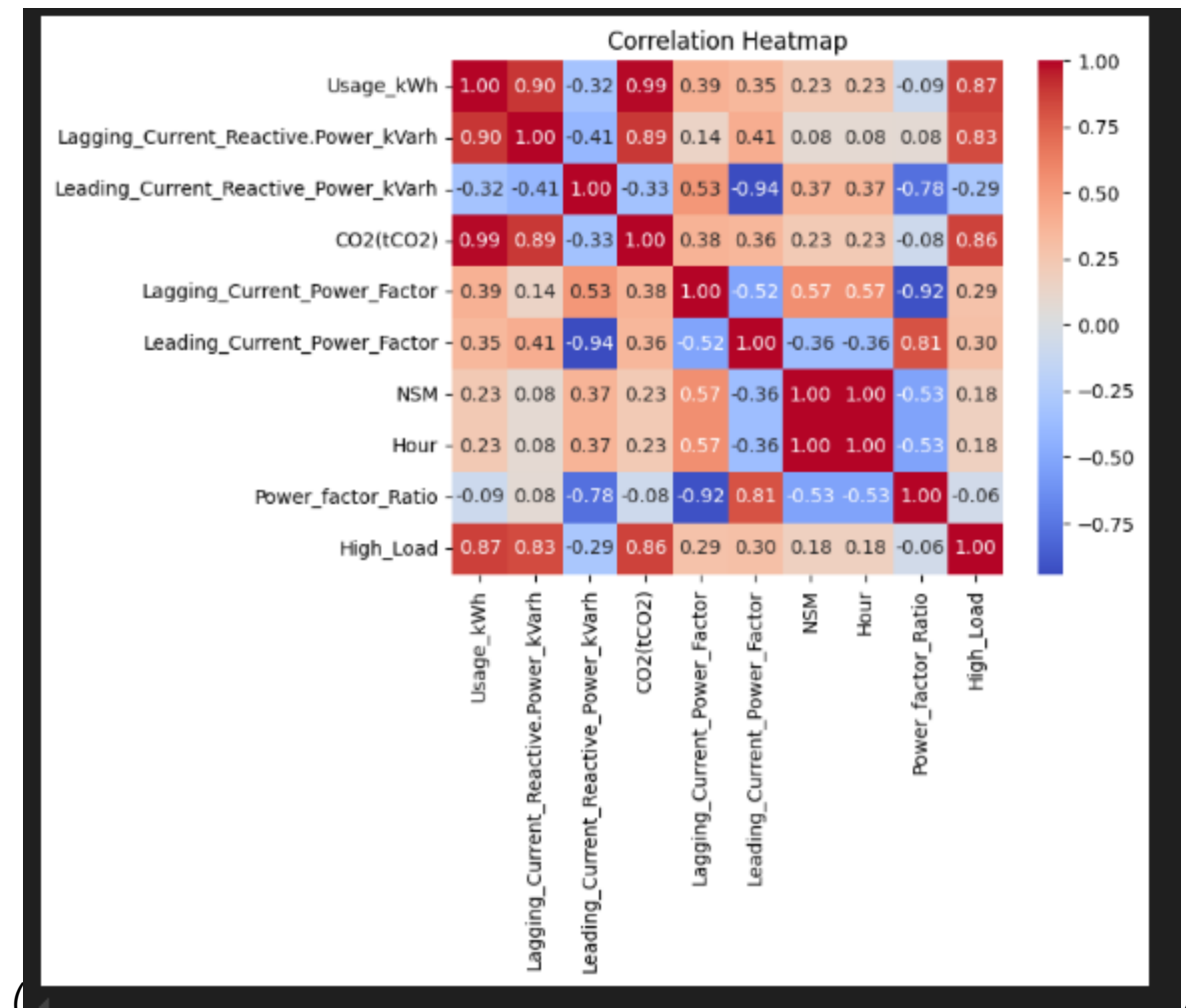
A correlation heatmap was created to measure the relationship between numerical variables.

The heatmap makes it easier to identify features that are strongly correlated with the target variable.

Explanation

Highly correlated features may have a greater influence on energy consumption prediction, while weakly correlated variables may contribute less to the model.

Screenshot:



5.3 Outlier Detection

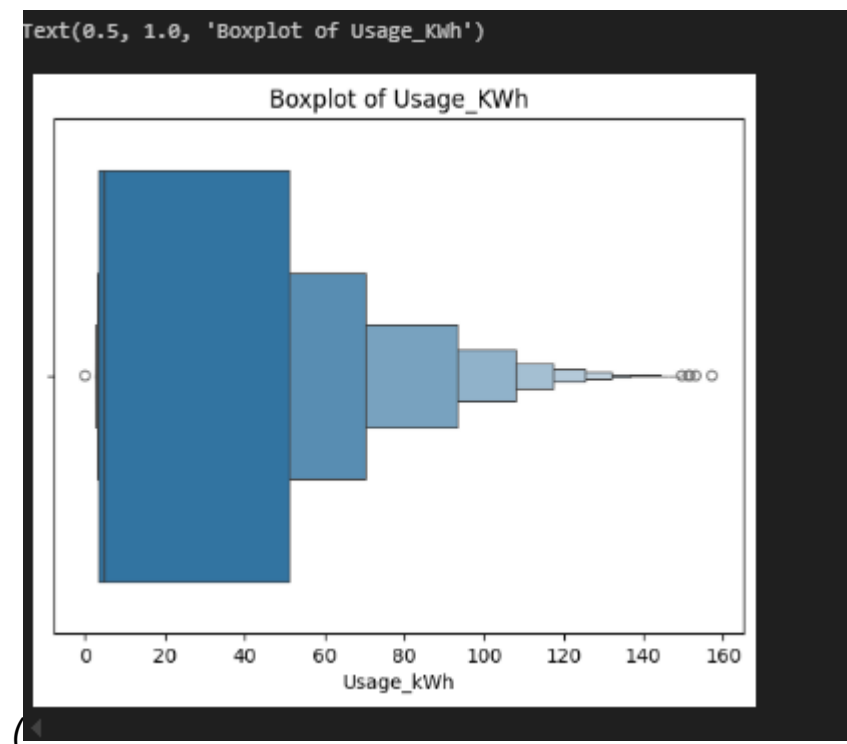
Box plots were used to detect outliers in the numerical features.

Outliers represent unusually high or low values that may influence model performance.

Explanation

The box plots helped identify extreme observations that required further investigation before model training.

Screenshot:



6. Feature Engineering

Feature engineering was performed to improve the quality of the dataset and enhance model performance.

Relevant features were selected, unnecessary variables were removed where required, and categorical variables were prepared for encoding.

Proper feature engineering allows machine learning algorithms to learn more effectively from the available data.

7. One-Hot Encoding

Machine learning algorithms require numerical input. Therefore, categorical variables were converted into numerical format using **One-Hot Encoding**.

One-Hot Encoding creates separate binary columns for each category, allowing the model to process categorical information without introducing false numerical relationships.

Observation

After applying One-Hot Encoding, all categorical features became suitable for regression modeling.

8. Feature Scaling

Feature Scaling was performed using **StandardScaler**.

Scaling transforms numerical features to a common scale by standardizing their values. This prevents variables with larger numerical ranges from dominating the learning process.

Feature scaling improves training efficiency and enhances the performance of regression algorithms.

9. Baseline Regression Modeling

After completing data preprocessing, the dataset was prepared for machine learning. The input features (X) and target variable (y) were separated. The dataset was then divided into training and testing sets using the **train_test_split()** function. Finally, regression algorithms were trained to predict steel industry energy consumption.

The primary objective of baseline regression modeling was to evaluate the predictive performance of different regression algorithms and identify the model that produced the most accurate results.

Screenshot:

```
(
Train test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size=0.2, random_state=42)

before apply linear regression model to use standardscaler for normalize value

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.fit_transform(X_test)
df.head()

Usage_kWh  Lagging_Current_Reactive_Power_kVarh  Leading_Current_Reactive_Power_kVarh  CO2(tCO2)  Lagging_Current_Power_Factor  Leading_Current_Power_Factor  NSM  Hour  Power_factor
0          3.17                               2.95                        0.0        0.0                73.21                100.0    900    0          1.0
1          4.00                               4.46                        0.0        0.0                66.77                100.0   1800    0          1.0
2          3.24                               3.28                        0.0        0.0                70.28                100.0   2700    0          1.0
3          3.31                               3.56                        0.0        0.0                68.09                100.0   3600    1          1.0
4          3.82                               4.50                        0.0        0.0                64.72                100.0   4500    1          1.0

5 rows x 30 columns
```

.)

10. Linear Regression

Linear Regression was used as the first baseline model for predicting energy consumption.

Linear Regression establishes a linear relationship between the independent variables and the target variable. It is one of the simplest and most widely used regression algorithms in machine learning.

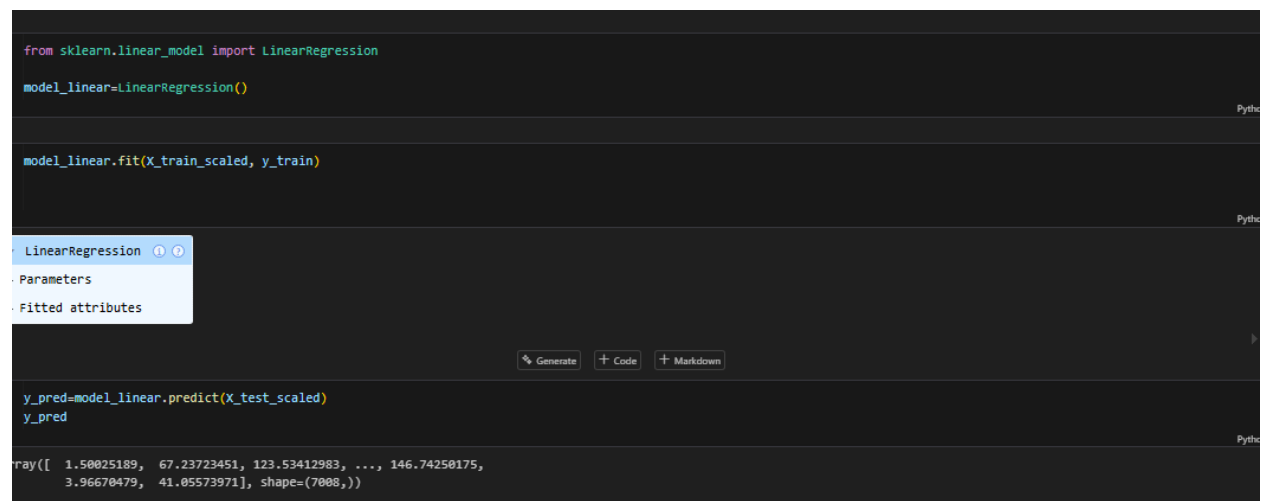
The model was trained using the training dataset and predictions were generated on the testing dataset.

Explanation

Linear Regression provides a strong baseline for comparison with other regression algorithms. It performs well when the relationship between input features and the target variable is approximately linear.

Screenshot:

(

A screenshot of a Jupyter Notebook interface. The top part shows a code cell with the following Python code:

```
from sklearn.linear_model import LinearRegression
model_linear=LinearRegression()

model_linear.fit(X_train_scaled, y_train)
```

 Below the code cell is a dropdown menu for the 'LinearRegression' object, showing options for 'Parameters' and 'Fitted attributes'. The bottom part shows the output of the model, which is a NumPy array of predictions:

```
y_pred=model_linear.predict(X_test_scaled)
y_pred
```

 The output is displayed as a NumPy array:

```
array([ 1.58025189,  67.23723451, 123.53412983, ..., 146.74250175,
        3.96670479,  41.05573971], shape=(7008,))
```

.)

11. Ridge Regression

Ridge Regression was implemented to improve the performance of Linear Regression by applying regularization.

This algorithm introduces a penalty term to the cost function, reducing the impact of large coefficients and minimizing overfitting.

The Ridge Regression model was trained using the scaled training dataset and evaluated on the testing dataset.

Explanation

Regularization improves the model's ability to generalize to unseen data by preventing excessive dependence on any single feature.

Screenshot:

```
from sklearn.linear_model import Ridge

ridge=Ridge(alpha=1.0)
ridge.fit(X_train_scaled,y_train)

prediction

ridge_pred = ridge.predict(X_test_scaled)

ridge_pred

array([ 1.50946939,  67.2468099, 123.53579557, ..., 146.73624698,
        3.96869883,  41.04593647], shape=(7008,))
```

(

12. Decision Tree Regression

Decision Tree Regression was used as another baseline model for predicting energy consumption.

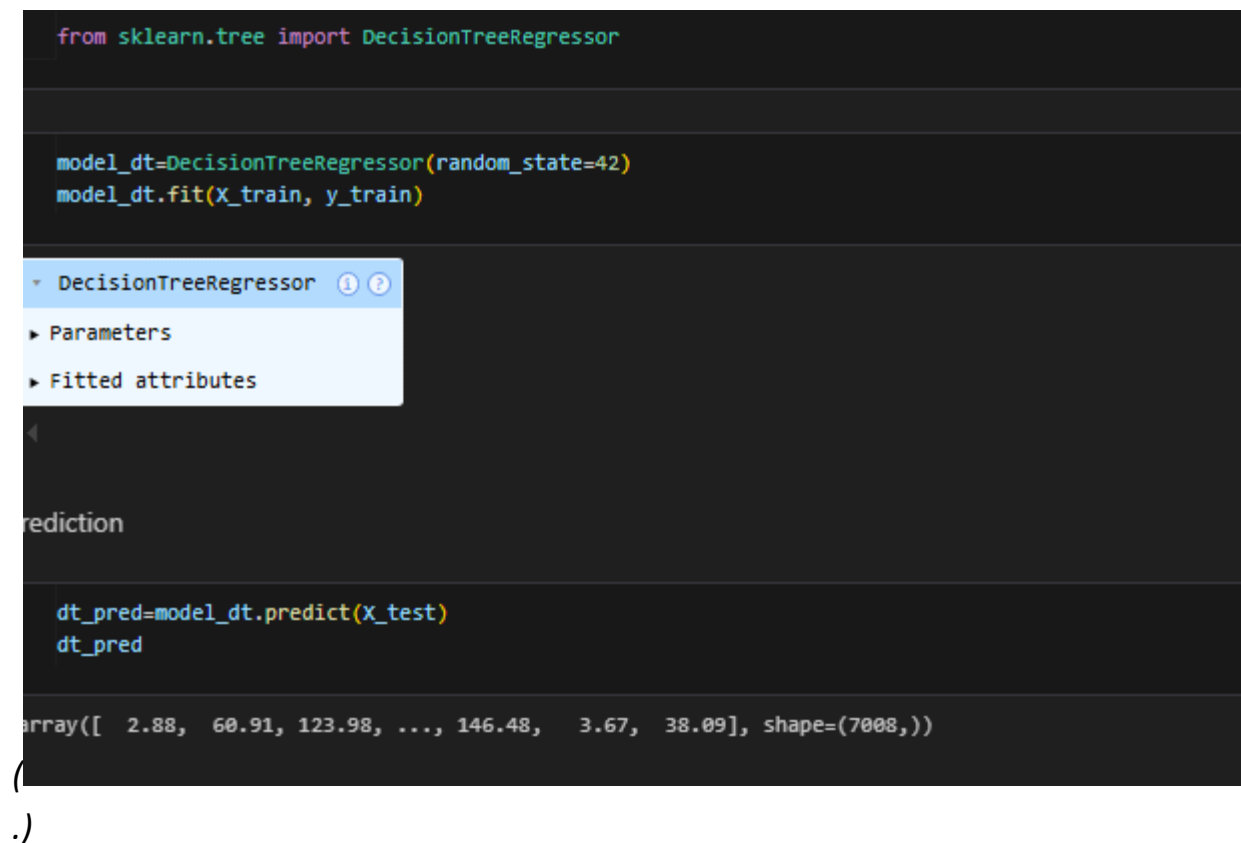
Unlike Linear Regression, Decision Tree Regression can capture complex non-linear relationships between input variables and the target variable.

The algorithm builds a tree-like structure by recursively splitting the dataset based on the most informative features.

Explanation

Decision Tree Regression can model complex patterns within industrial datasets and often provides higher prediction accuracy than simple linear models.

Screenshot:



```
from sklearn.tree import DecisionTreeRegressor

model_dt=DecisionTreeRegressor(random_state=42)
model_dt.fit(X_train, y_train)

DecisionTreeRegressor ⓘ ⓘ
  Parameters
  Fitted attributes

prediction

dt_pred=model_dt.predict(X_test)
dt_pred

array([ 2.88, 60.91, 123.98, ..., 146.48,  3.67, 38.09], shape=(7008,))

(
.)
```

13. Model Evaluation

The performance of all regression models was evaluated using standard regression metrics.

These evaluation metrics provide quantitative measurements of prediction accuracy and help determine the best-performing model.

13.1 Mean Absolute Error (MAE)

Mean Absolute Error (MAE) measures the average absolute difference between the actual values and the predicted values.

A lower MAE indicates that the predictions are closer to the actual observations.

13.2 Mean Squared Error (MSE)

Mean Squared Error (MSE) calculates the average squared difference between actual and predicted values.

Because the errors are squared, larger prediction errors receive greater penalties.

A lower MSE indicates better model performance.

13.3 Root Mean Squared Error (RMSE)

Root Mean Squared Error (RMSE) is the square root of the Mean Squared Error.

RMSE is easier to interpret because it is expressed in the same unit as the target variable.

A smaller RMSE indicates a more accurate prediction model.

13.4 R² Score

The R² Score measures how well the regression model explains the variation in the target variable.

The R² value ranges from 0 to 1.

A value close to **1** indicates excellent prediction performance.

14. Model Comparison

The performance of all regression models was compared using the evaluation metrics.

The comparison helped determine which algorithm achieved the highest prediction accuracy and the lowest prediction error.

Model	MAE	MSE	RMSE	R ² Score
Linear Regression	2.74331584	17.410553	4.1727	0.984669
Ridge Regression	2.740	17.404	4.11	0.9846
Decision Tree Regression	0.541	2.3928	1.546	0.997
Random forest regression	0.345	1.062	1.03	0.99906

Observation

Based on the evaluation metrics, the model with the highest R² Score and the lowest error values was considered the best-performing regression model for predicting steel industry energy consumption.

